GameDay Stats – Brainstorming and Code Planning

1. Original Idea

For my final project, I wanted to create a website that would be useful for athletes and coaches. Since I play sports, I thought it would be interesting to make a website where people could enter player statistics and keep track of how players are performing throughout a season.

I decided to call my website **GameDay Stats**. The main goal is to make it easier to organize player information, view statistics, and compare players without having to calculate everything manually.

2. Brainstorming Ideas

Before deciding on my final idea, I thought about different features that could make the website useful.

Some ideas I came up with were:

- Adding players to a team roster.
- Choosing which sport each player participates in.
- Entering player statistics.
- Viewing all players on one roster page.
- Editing statistics if they change.
- Deleting players from the roster.
- Comparing players using a leaderboard.
- Viewing team statistics.

I decided to include basketball, soccer, and softball because they are all sports where keeping track of statistics can be useful.

3. Planning the Website Pages

I wanted the website to be simple to navigate, so I planned out different pages for each feature.

Home Page

- Shows the website name and team name.
- Gives users options to add a player or view statistics.

Add Player Page

- Allows users to enter a player's name.
- Allows users to select a sport.
- Allows users to enter points, assists, rebounds, and games played.

Team Roster Page

- Displays all players that have been added.
- Allows users to click on a player to see more information.

Player Details Page

- Shows the selected player's information.
- Gives users the option to edit or delete the player.

Statistics Page

- Shows team statistics.
- Helps users understand the team's overall performance.

Leaderboard Page

- Allows users to compare players.
- Shows which player has the highest number in a selected statistic category.

4. Planning My Data Abstractions

Before writing the code, I needed to figure out what information the website would have to store. I decided to use two dataclasses.

Player Dataclass

Each player needs their own information, so I planned to include:

- player_id
- name
- sport
- points
- assists
- rebounds
- games_played

The Player dataclass will keep all of one player's information together instead of having to store everything separately.

State Dataclass

I also needed a State dataclass to keep track of the overall website information.

The State dataclass will include:

- players: stores the list of players.
- selected_sport: stores the sport selected by the user.
- team_name: stores the team's name.
- message: displays messages to the user.
- next_player_id: keeps track of the next player ID.

5. Planning the Code

I planned to use different functions, routes, loops, and conditional statements to make the website work.

Some of the main functions and routes I planned were:

- `index()` – Displays the home page.
- `add_player()` – Shows the form for entering a new player.
- `save_player()` – Saves the player's information.
- `view_players()` – Displays the team roster.
- `player_details()` – Shows information about one player.
- `edit_stats()` – Allows the user to change a player's statistics.
- `update_stats()` – Saves the updated statistics.
- `delete_player()` – Removes a player from the roster.
- `view_stats()` – Displays team statistics.
- `leaderboard()` – Compares players.

I also planned to create a helper function called `find_player()` to find a player using their ID. This would make it easier to edit, delete, and display individual players without repeating the same code.

6. Planning the Programming Concepts

I wanted to make sure I used the programming concepts we learned in class.

Lists: I will use a list to store all the Player objects in the team roster.

Loops: I will use loops to go through the player list when displaying the roster, calculating statistics, and comparing players.

If Statements: I will use conditional statements to check if a player name is blank, if statistics are negative, or if a player exists in the roster.

Functions: I will use separate functions for different parts of the website so the code is organized and easier to understand.

Dataclasses: I will use Player and State dataclasses to organize the information being stored.

7. Possible Problems and How I Planned to Solve Them

One part I thought might be difficult was figuring out how to store all the player information and make sure it stayed updated when someone edited or deleted a player.

Another challenge was figuring out how to calculate statistics and compare players using loops and functions.

To make this easier, I planned to build the website one part at a time. I would start with the dataclasses and the ability to add players. After that, I would work on displaying the roster, editing and deleting players, and finally adding the statistics and leaderboard.

I also planned to test each feature as I added it so I could find errors before moving on to the next part.

8. Final Goal

My goal for GameDay Stats is to create a website that is easy to use and helps athletes and coaches organize player statistics. I also want this project to help me improve my understanding of Python, especially using dataclasses, lists, loops, functions, and conditional statements together.

I wanted to create something that has a real purpose and connects to something I am interested in, which is sports.

Planning:

Route	Purpose
`index`	Displays the home page.
`update_team`	Open the page to change the team name.

`save_team_name`	Saves the new team name.
`add_player`	Displays the form to add a player.
`save_player`	Saves a player's information.
`view_players`	Displays the team roster.
`player_details`	Displays information about one player.
`edit_stats`	Shows the form to edit player statistics.
`update_stats`	Saves updated statistics.
`delete_player`	Removes a player from the roster.
`view_stats`	Displays team statistics.
`leaderboard`	Compares players in a selected category.

I planned different routes so each part of the website would have its own purpose. Some routes display pages, while others save information or make changes to the roster. This helps keep my code organized and makes it easier to connect the different pages

Title: Planning My Dataclasses
I decided to use two dataclasses to organize the information for my website.
Player Dataclass

Attribute	Data Type	Purpose
`player_id`	Integer	Gives each player a unique ID.
`name`	String	Stores the player's name.
`sport`	String	Stores the player's sport.
`points`	Integer	Stores the player's points.

`assists`	Integer	Stores the player's assists.
`rebounds`	Integer	Stores the player's rebounds.
`games_played`	Integer	Stores the number of games played.

State Dataclass

Attribute	Data Type	Purpose
`players`	List of Players	Stores all players added to the website.
`selected_sport`	String	Stores the selected sport.
`team_name`	String	Stores the team name.
`message`	String	Displays messages to the user.
`next_player_id`	Integer	Keeps track of the next player ID.

Dataclass:

```
@dataclass
class Player:
    player_id: int
    name: str
    sport: str
    points: int
    assists: int
    rebounds: int
    games_played: int
```

State dataclass:

```
@dataclass
class State:
    players: list[Player] = field(default_factory=list)
    selected_sport: str = "Basketball"
    team_name: str = "My Team"
    message: str = ""
    next_player_id: int = 1
```

I used the Player dataclass because I needed a way to keep each player's information together. I used the State dataclass to store the overall website information. This made it easier to organize the data and access it from different routes.

Steps

Step	What I planned to do
1	**Come up with the website idea and decide on the features.**
2	**Create the Player and State dataclasses.**
3	**Build the home page and navigation buttons.**
4	**Create the Add Player page and save player information.**
5	**Create the roster and player details pages.**
6	**Add the ability to edit and delete players.**
7	**Create the statistics and leaderboard pages.**
8	**Test the website and fix errors.**

How should I organize my different Drafter routes so each page connects to the correct page?

For my GameDay Stats website, I would organize my Drafter routes by giving each page its own function. Each route should have one main purpose, and buttons should connect the pages so users can move through the website without getting lost.

I would organize the routes in the following order:

1. Home page

The `index()` route should be the starting point of the website. It should have buttons that take users to the Add Player page, Team Roster, and Statistics page.

2. Adding players

- `add_player()` displays the form where users enter a player's name, sport, and statistics.
- `save_player()` takes the information from the form and adds the player to the list in the State dataclass.

3. Viewing players

- `view_players()` displays all the players that have been added.
- `player_details()` displays the information for one selected player.

4. Editing and deleting players

- `edit_stats()` displays the form to change a player's statistics.
- `update_stats()` saves the updated information.
- `delete_player()` removes a player from the roster.

5. Viewing statistics

- `view_stats()` displays the team's statistics.
- `leaderboard()` compares players based on a selected statistic, such as points or assists.

How the routes connect

Here is an example of how a button connects one route to another:

Python Run

```
Button("Add Player", "add_player")
```

This button takes the user to the `add_player()` route.

If I want to send information about a specific player to another route, I can use an `Argument`:

Python Run

```
Button(
    "View Details",
    "player_details",
    [Argument("player_id", player.player_id)]
)
```

This sends the selected player's ID to the `player_details()` route so the website knows which player to display.

How do I give every player a unique ID when they are added to the roster?

The easiest way to give every player a unique ID is to use the `next_player_id` variable in my **State dataclass**. This variable keeps track of the next available ID so that every time a new player is added, they receive a different number.

1. Add the ID tracker to the State dataclass

Python Run

```
@dataclass
class State:
    players: list[Player] = field(default_factory=list)
    selected_sport: str = "Basketball"
    team_name: str = "My Team"
    message: str = ""
    next_player_id: int = 1
```

I start `next_player_id` at 1 because the first player added will receive ID 1.

```
state.players.append(new_player)
state.next_player_id += 1
```

3. How it works

For example:

Player	Assigned ID
Sarah	1
Emily	2
Jessica	3
Amanda	4

Every time a player is saved, the ID increases by one. Even if a player is deleted, the next player will receive the next available number instead of reusing an old ID.

How do I create a home page in Drafter with buttons that take users to other pages?

To create the home page for my GameDay Stats website, I would use the `index()` route. In Drafter, the `index()` route is the main page that users see when they first open the website.

I want my home page to be simple, with the website title, team name, and buttons that take users to the other pages.

1. Create the index route

First, I would create a function called `index()` and use `Page()` to display the content.

Python Run

```python
@route
def index(state: State) -> Page:
    return Page(state, [
        Header("GameDay Stats"),
        Text("Welcome to GameDay Stats!"),
        Text("Team: " + state.team_name),

        Button("Add Player", "add_player"),
        Button("View Team Roster", "view_players"),
        Button("View Statistics", "view_stats"),
        Button("Leaderboard", "leaderboard")
    ])
```

2. How the buttons work

Each button has two important parts:

- The first part is the text that appears on the button.
- The second part is the name of the route the button should connect to.

How do I take the information entered into the form and save it as a Player object?

To save the information entered into my Add Player form, I need to connect the form to a route called `save_player()`. This route takes the information entered by the user, creates a new `Player` object, and adds it to the players list in my State dataclass.

1. Create the Add Player form

First, I need to create a page with input boxes where users can enter the player's information.

3. Understand what happens

There are a few important steps in this code:

1. The user enters the player's information into the form.
2. Drafter sends the information to the `save_player()` route.
3. The if statements check that the name is not blank and the statistics are not negative.
4. A new `Player` object is created using the information entered.
5. `.append()` adds the new player to the `state.players` list.
6. `next_player_id` increases so the next player gets a different ID.
7. The user is sent to the Team Roster page.

Can you help me write a function that searches through my player list and finds a player by their ID?

Yes! For GameDay Stats, I would create a helper function called `find_player()`. Its job is to search through the list of players and return the player that matches the ID entered.

This is useful because I need to find individual players when someone wants to view their details, edit their statistics, or delete them.

2. How the code works

- `def find_player()` creates the function.
- `state` gives the function access to the website's information.
- `player_id` is the ID of the player I am searching for.
- The `for` loop goes through every player in the list.
- The `if` statement checks whether the player's ID matches the ID I am looking for.
- `return player` gives back the matching player.
- `return None` means the player could not be found.